Explorer-KQL: An in-form Application Insights query editor for Dynamics 365

Mission Critical · Dataverse · PCF · Application Insights · KQL

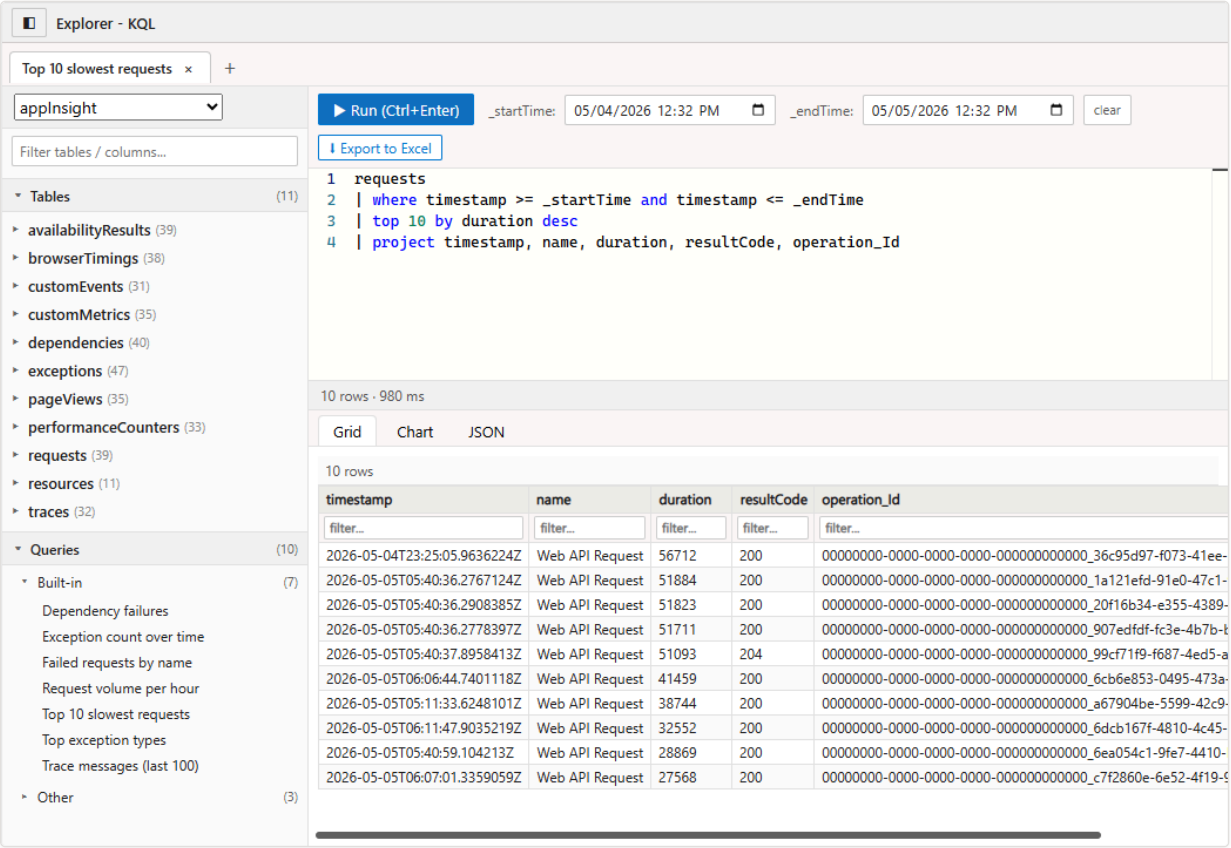
Source code: github.com/SweetsNSavories/Explorer-KQL · **Managed solution:** v1.0.3 zip · **License:** MIT

> **Disclaimer.** This is a personal, open-source project. It is **not a Microsoft product, not endorsed by Microsoft, and not supported through any Microsoft support channel.** The author works at Microsoft; this work represents the author's own opinions and experiments and does not reflect those of the employer. The implementation follows generally recommended Microsoft best-practice guidance for Power Platform / Dataverse / Azure — least-privilege security roles, Secret-type environment variables, signed sandboxed plugins, public REST endpoints documented on Microsoft Learn — but no part of it is officially reviewed, certified, or endorsed by Microsoft. Use at your own risk under the terms of the MIT License.

Application Insights is increasingly where the diagnostic *truth* lives for a Dataverse application. Plugin trace logs, Power Automate run history, custom telemetry, and now **Conversation Diagnostics** for Copilot Studio / agent flows all flow into Application Insights — the surface that used to live inside Dataverse has effectively moved to Azure Monitor.

Most organizations do provision the Azure subscription and the Application Insights resource — that part is rarely the blocker. The blocker is granting *per-user RBAC* on it, and then asking a business owner to leave the model-driven app they live in, sign in to the Azure portal, find the right resource, and learn the Logs blade just to answer a question about *their own* application. From a UX standpoint that's overwhelming — it feels like "that's not mine, that's some ops team's Purview-shaped territory', and the user disengages.

Explorer-KQL inverts that. Azure credentials live *once* in a single Dataverse-side service principal (held by the plugin), and any user the admin trusts gets a full KQL editor — tabs, schema tree, saved queries, time pickers, grid/chart/JSON output, Excel export — right inside the model-driven form they already use every day. No Azure portal hop, no per-user RBAC, no context switch. Putting just the relevant slice of telemetry inside the user's familiar tool invites them to use it, rather than alienating them.



Explorer-KQL embedded on a Dataverse form, running a built-in saved query against Application Insights.

In this post

1. What it does
2. Architecture

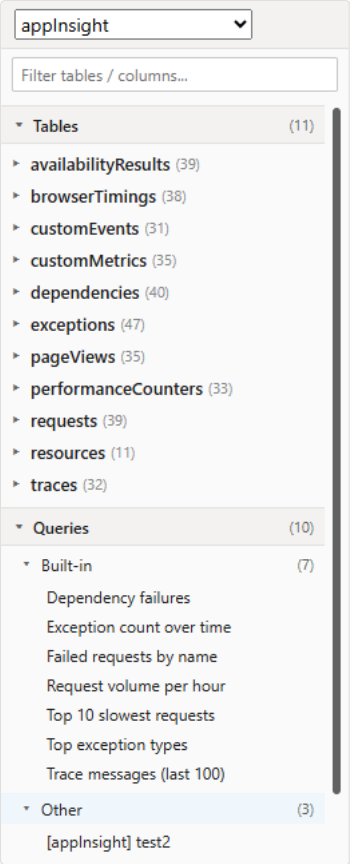
3. Inside the plugin (NuGet, AAD, REST)
4. Custom form & security role
5. Prerequisites
6. Install the solution
7. Configure environment variables
8. Grant Azure permissions
9. Using the control
10. Troubleshooting

What it does

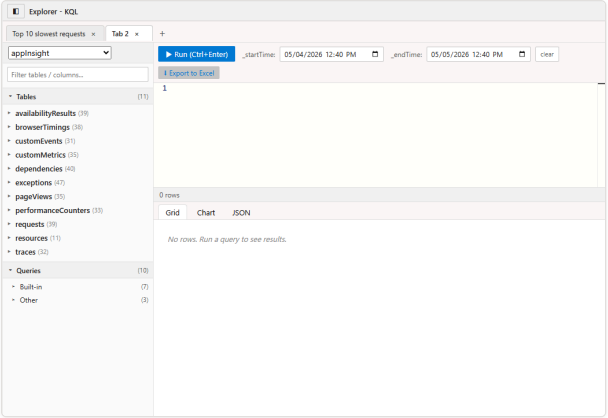
Explorer-KQL is a virtual PCF (React) control that talks to Application Insights through a Dataverse Custom API (`vip_azuremonitorquery`). The Custom API is implemented by a server-side plugin that holds the Azure AD service principal credentials and proxies four operations:

- `apps` — lists the configured Application Insights instances.
- `schema` — returns the table/column metadata for a given app.
- `savedqueries` — returns built-in queries plus everything from the app's *analyticsItems* ("Other") and any *queryPacks* in the same resource group (Alerts, Performance, Browsing data, etc.).
- `query` — runs an arbitrary KQL string and returns a row array.

Because the credentials live in the plugin, end users *never* need direct Azure access — they get full KQL diagnostics with only their existing Dataverse role.



Tables & Queries are unified in a single accordion sidebar.



Each tab is fully independent — its own AI instance, time window, and query.

Architecture

The end-to-end request path is intentionally short:

```
┌──────────────────────────┐
│  PCF control (React/TS)  │  Xrm.WebApi.execute({ vip_azuremonitorquery })
```

```
│  in the model-driven form│ ──────────────────────────────────────────┐
└──────────────────────────┘                                            ▼
                                                        ┌──────────────────────────┐
                                                        │  Custom API: vip_azuremonitorquery │
                                                        │  (Stage 30 plugin step)           │
                                                        └──────────────────────────┘
                                                                    │
                                                                    │  https
                                                                    ▼
                                                  ┌──────────────────────────────────────┐
                                                  │  login.microsoftonline.com  /token      │
                                                  │  (client-credentials, SP)               │
                                                  └──────────────────────────────────────┘
                                                                ▼                ▼
                                          api.applicationinsights.io  management.azure.com
                                          (query / metadata)          (queryPacks, analyticsItems)
```

- The control never holds Azure credentials. It speaks *only* to a Dataverse Custom API, just like any other Web API call.
- The plugin is the trust boundary — it owns the service principal, mints AAD tokens, and proxies the call.
- Two AAD scopes are used: `https://api.applicationinsights.io/.default` for the data plane (KQL + metadata), and `https://management.azure.com/.default` for the control plane (query packs & analyticsItems lists).

Inside the plugin (NuGet, AAD, REST)

Why no Azure.Monitor.Query SDK?

The natural instinct is to `dotnet add package Azure.Monitor.Query` and call it a day — but the Dataverse plugin **sandbox** blocks almost every transitive dependency that modern Azure SDK clients need (System.Memory, System.Buffers, MSAL, dependency-injection helpers, etc.). Even with ILMerge / ILRepack, the sandbox's strict assembly-load rules and isolation-level reflection checks reject most of them at runtime.

The pragmatic answer is to keep the plugin's reference graph **tiny** and call the same REST endpoints the SDK would call under the hood. The full `.csproj` is just two NuGet packages plus the BCL `System.Net.Http`:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net462</TargetFramework>
    <LangVersion>latest</LangVersion>
    <SignAssembly>true</SignAssembly>
    <AssemblyOriginatorKeyFile>vip.snk</AssemblyOriginatorKeyFile>
    <AssemblyName>vip.AzureMonitorQuery</AssemblyName>
    <RootNamespace>vip.AzureMonitor</RootNamespace>
    <CopyLocalLockFileAssemblies>false</CopyLocalLockFileAssemblies>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.CrmSdk.CoreAssemblies" Version="9.0.2.54" />
    <PackageReference Include="Newtonsoft.Json"               Version="13.0.3"  />
    <Reference          Include="System.Net.Http" />
  </ItemGroup>
</Project>
```

- **net462** — Dataverse plugin sandbox runtime.
- **SignAssembly** — Dataverse refuses unsigned plugin assemblies. `vip.snk` is generated once with `sn -k vip.snk`.
- **CopyLocalLockFileAssemblies=false** — we ship a single DLL; transitive assemblies stay out of the artifact.
- **Microsoft.CrmSdk.CoreAssemblies** — gives you `IPlugin`, `IOrganizationService`, the `QueryExpression` family, etc.
- **Newtonsoft.Json** — the only JSON parser available in the sandbox; pre-loaded by the host.

Acquiring an AAD token without MSAL

Without MSAL, we POST a plain `client_credentials` form to the v2 token endpoint. Tokens are cached in-memory inside `TokenService` (keyed by `clientId|tenantId|scope`) so a warm sandbox reuses them across calls until expiry:

```
POST https://login.microsoftonline.com/<tenant>/oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
```

```
&client_id=<app-id>
&client_secret=<secret>
&scope=https://api.applicationinsights.io/.default
```

Talking to Application Insights

For data- and control-plane operations the plugin calls the same first-party REST endpoints that the official Azure SDKs are generated from. Every URL below is publicly documented — no reflection, no decompilation required.

Operation	Endpoint	Auth scope	R
`query`	`POST /v1/apps/{appId}/query`	`api.applicationinsights.io/.default`	R —
`schema`	`GET /v1/apps/{appId}/metadata`	`api.applicationinsights.io/.default`	R M G
`savedqueries` (per-app legacy)	`GET {armId}/analyticsItems?api-version=2015-05-01&scope=shared&type=query&includeContent=true`	`management.azure.com/.default`	A A —
`savedqueries` (query packs)	`GET /subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Insights/queryPacks?api-version=2019-09-01-preview` + per-pack `/queries`	`management.azure.com/.default`	A P P

The plugin sandbox — things to know

- **No filesystem, no `HttpClientHandler` tweaks.** Use a *static* `HttpClient` instance. Re-creating one per call leaks sockets fast in long-running app domains.
- **No async-await all the way down.** The plugin entry point is synchronous, so we call `.GetAwaiter().GetResult()` at the boundary. Only do this on truly synchronous code paths.
- **No environment variables via `Environment.GetEnvironmentVariable`.** Read Dataverse environment variables through the SDK (`environmentvariabledefinition` + `environmentvariablevalue`). The helper `ReadEnvVar` in the plugin does exactly that.
- **Trace early, trace often.** The plug-in trace log is your only window into a sandbox failure. Every external call in this plugin logs status codes and (truncated) bodies on failure.

Build & deploy commands

```
# 1. Build the plugin (NuGet restore + compile to a single signed DLL)
cd PreOpPlugin
dotnet build -c Release

# Output: bin\Release\net462\vip.AzureMonitorQuery.dll  (~30 KB)

# 2. Push the DLL into Dataverse via Web API (no Plugin Registration Tool needed)
cd ..\deploy
node push-plugin.mjs           # PATCH pluginassembly + PublishXml

# 3. Build & push the PCF (React/TS bundle)
cd ..\KustoExplorerControl
pac pcf push --publisher-prefix vip

# 4. Re-export the whole thing as a managed solution zip
cd ..\deploy
node export-solution.mjs       # writes dist\KustoExplorerSolution_*_managed.zip
```

> Total plugin DLL size: **~30 KB**. No ILMerge, no shadow copies of MSAL or Azure.Identity, nothing that the sandbox can refuse to load.

References — OpenAPI specs & official docs

The two REST surfaces the plugin uses are described by Microsoft's public OpenAPI / Swagger specifications. The official Azure SDKs (.NET `Azure.Monitor.Query`, JavaScript `@azure/monitor-query`, Java, Go, etc.) are *generated* from these same specs — so calling the URLs directly is exactly what those SDKs do under the hood.

- **Application Insights data plane** (KQL query + metadata) — OpenAPI: azure-rest-api-specs → applicationinsights / data-plane. Service overview: api.applicationinsights.io overview.
- **Application Insights control plane** (analyticsItems on the component) — OpenAPI: azure-rest-api-specs → applicationinsights / resource-manager.
- **Log Analytics Query Packs** (cross-workspace shared queries) — OpenAPI: azure-rest-api-specs → operationalinsights / resource-manager. Concept: Query packs in Azure Monitor Logs.
- **AAD client-credentials token endpoint** — OAuth 2.0 client credentials flow (Microsoft Entra ID).
- **Dataverse Custom API** (the contract this plugin implements) — Create and use Custom APIs.

Custom form & security role

The "Kusto Explorer" form on `systemuser`

The control runs on a **new, dedicated form** on the `systemuser` entity called "Kusto Explorer" (`formid = 69c22e59-1888-4d06-9afb-4d301a3a5d2f`). Two design choices matter here:

- **It is a *new* form, not an edit of the OOB User form.** The solution exports the systemuser entity with `behavior="2"` (metadata-only), which means existing Main / Quick Create / Card forms are left exactly as they are. Only the new "Kusto Explorer" form is added.
- **Why systemuser?** Every Dataverse environment already has a User table and a User form selector in the model-driven app. By piggy-backing on it, Explorer-KQL does *not* introduce a new table, a new sitemap area, or a new app — an admin opens any user record, switches to "Kusto Explorer", and the PCF takes over the full canvas.

The form itself is intentionally minimal: a single full-width section that hosts the PCF (`vip_vip.KustoExplorer`) bound to a no-op attribute, plus the form-script web resource `vip_/js/kustoexplorer_form.js` which hides the rest of the command bar and stretches the control to the viewport.

The "PPAC Kusto Reader" role

Shipping the PCF without a role would force admins to invent one or grant System Administrator. Instead the managed solution ships a least-privilege role — **PPAC Kusto Reader** (`roleid = 08d61102-7247-f111-bec6-7c1e5267f8c3`) — designed so that an operator with *only* this role plus the standard Basic User role can open the Kusto Explorer form and execute queries.

Privilege grid (all granted at **Global** scope):

Privilege	Why it's needed
`prvReadUser`	Open `systemuser` records so the form can be displayed.
`prvReadEntity`	Read entity metadata so the model-driven host can render the form.
`prvReadCustomization`	Load the form definition, the PCF manifest, and the web resource binding.
`prvReadWebResource`	Download `vip_/js/kustoexplorer_form.js` and the PCF bundle.
`prvReadCustomAPI`	Discover and call the `vip_azuremonitorquery` Custom API. (Custom API execution is governed by read on the message + ownership of the bound entity; no separate *execute* privilege exists.)
`prvReadSdkMessage` , `prvReadSdkMessageProcessingStep` , `prvReadSdkMessageProcessingStepImage`	Required by the platform when invoking a Custom-API-registered SDK message from the Web API.
`prvReadPluginAssembly` , `prvReadPluginType`	Allow the platform to resolve the plugin that backs the Custom API.
`prvReadEnvironmentVariableDefinition`	Lets the plugin (running in the caller's context) resolve `vip_AppInsightsAppId / TenantId / ClientId / ClientSecret` .
`prvReadOrganization`	Standard read of org settings used by the form runtime.
`prvReadSharePointData` , `prvReadSharePointDocument` , `prvCreateSharePointData` , `prvWriteSharePointData`	Carry-over from the role template; harmless if SharePoint integration is not enabled, and ignored otherwise. Safe to remove post-import if your governance bars them.

> **What the role does NOT grant.** No write/create/delete on any business data. No access to plugin *registration* (only read of the metadata so the message can be invoked). No access to environment variable *values* beyond what the plugin needs to resolve them on the caller's behalf. No System Administrator equivalence.

> **How to assign.** In the Power Platform admin center, open the target environment → Users → pick the user → Manage security roles → check *PPAC Kusto Reader* alongside the user's base role (typically Basic User). The user can now navigate to any `systemuser` record, switch to the "Kusto Explorer" form, and run queries.

Prerequisites

Component	Required	Notes
Dataverse environment	Any modern (9.2+) production or sandbox	You need a System Customizer / System Administrator role to import the solution.
Maximum upload file size	≥ 32 MB	The PCF bundle is large. If `maxuploadfilesize` is the default 16 MB you'll see `0x8004f114` on import. Bump it via Power Platform admin center → Environment → Settings → Email → *Maximum file size*, or via the Web API: `PATCH /api/data/v9.2/organizations({orgId})` `{ "maxuploadfilesize": 131072000 }`
Application Insights resource(s)	1 or more	Workspace-based or classic; both work. You'll need each app's **Application ID** (a GUID) and **Resource ID** (the full ARM path).
Microsoft Entra application (service principal)	Yes	Used by the plugin for non-interactive AAD auth. You'll capture *Tenant ID*, *Client ID*, and a *Client secret*.
Azure RBAC on each AI resource	`Monitoring Reader`	Granted to the SP above. This is what authorizes both the analytics query API and the Query Pack listing.

Install the solution

1. Download `KustoExplorerSolution_1_0_3_managed.zip` (attached to this post or from GitHub releases).
2. In *make.powerapps.com*, choose your environment, then **Solutions** → **Import solution** and upload the zip.
3. Accept the consent prompts. The import will install:
   - The `Explorer-KQL` custom control (`vip_vip.KustoExplorer`).
   - The `vip_azuremonitorquery` Custom API and its plugin assembly.
   - 4 environment variable **definitions** (the import wizard will ask you for their values — see next section).
   - The *Kusto Explorer* form on `systemuser` and the *PPAC Kusto Reader* security role.

> **The shipped solution carries no environment-variable values.** Definitions ship; values do not. The maker-portal import wizard will prompt you for each value before completing the import. If you skip them the plugin will fail at runtime when it tries to read the secret — that is the intended fail-fast behavior. No tenant's credentials are ever baked into the published artifact.

Configure environment variables

All non-secret configuration is held in environment variables so the same managed solution can ship across DEV / TEST / PROD.

Schema name	Type	Description
`vip_AppInsightsTenantId`	Text	The Microsoft Entra tenant GUID that owns your service principal.
`vip_AppInsightsClientId`	Text	Application (client) ID of the SP.
`vip_AppInsightsClientSecret`	Secret	Client secret value. **Never** the secret ID. In Vault-backed environments this points to a Key Vault secret URI; otherwise it is encrypted at rest by Dataverse.
`vip_AppInsightsAppId`	Text (JSON	The list of Application Insights instances the control should expose. **Use the JSON array form below** — a plain GUID also works for a single app, but loses friendly names and

Schema name	Type	Description
	array)	the Query Pack lookup.

Set `vip_AppInsightsAppId` to a JSON array, one entry per AI resource:

```
[
  {
    "name":  "DynamicsOfficeWebhook",
    "appId": "00000000-0000-0000-0000-000000000000",
    "armId": "/subscriptions/<sub-guid>/resourceGroups/<rg-name>/providers/Microsoft.Insights/components/<ai-name>"
  },
  {
    "name":  "appInsight",
    "appId": "11111111-1111-1111-1111-111111111111",
    "armId": "/subscriptions/<sub-guid>/resourceGroups/<rg-name>/providers/Microsoft.Insights/components/<ai-name>"
  }
]
```

- **name** — the friendly label shown in the per-tab AI selector.
- **appId** — the GUID under *Application Insights → Configure → API Access → Application ID*.
- **armId** — the full ARM resource ID. Required for the *Saved Queries* tree (analyticsItems + queryPacks). Without it, only the bundled built-in queries appear.

> **Heads-up:** Environment variables have both a default value (lives in the solution) and a current value (per environment). Set the current value, not the default — that way upgrading the managed solution doesn't overwrite per-environment configuration.

Grant Azure permissions

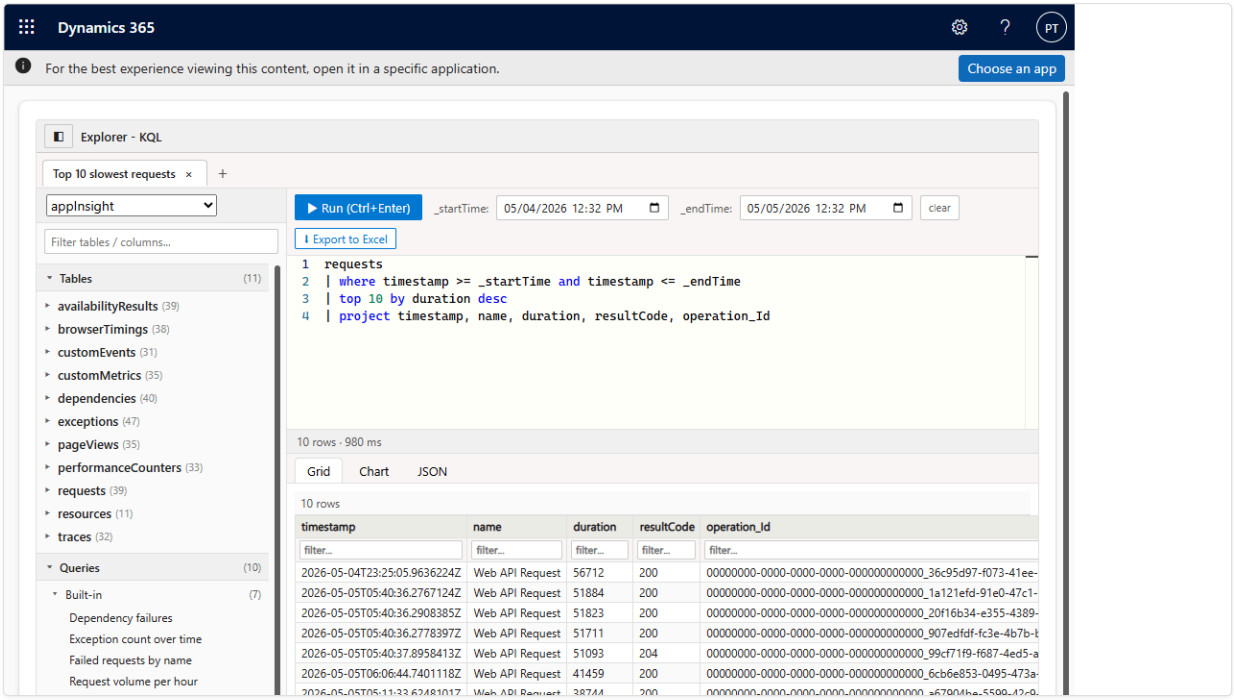
For each Application Insights resource you listed in `vip_AppInsightsAppId`:

1. Open the AI resource in the Azure portal.
2. Go to **Access control (IAM) → Add role assignment**.
3. Role: `Monitoring Reader`.
4. Members: search for the service principal by its *Object ID* or app display name (the same SP that backs `vip_AppInsightsClientId`).
5. **Save.** Wait 30–60 seconds for the role to propagate.

If you prefer the CLI:

```
az role assignment create \
  --role "Monitoring Reader" \
  --assignee <sp-object-id> \
  --scope "/subscriptions/<sub-guid>/resourceGroups/<rg-name>/providers/Microsoft.Insights/components/<ai-name>"
```

Using the control



Full layout: tabs, sidebar with Tables & Queries, time-window controls, Monaco editor, and result grid.

Tabs

- Click `+` to open a new tab. Each tab keeps its own KQL, time window, AI instance selection, and result set.
- Double-click a tab to rename.
- Click `×` to close it.

Sidebar

- **Tables**: live schema for the active tab's AI instance. Single-click to expand columns; double-click any table or column to insert it at the cursor.
- **Queries**: built-in templates plus every saved query & query-pack query you can see in the Azure portal's *Logs → Queries* pane. Categorized exactly the way the portal categorizes them (Alerts, Performance, Browsing data, Reports failures, Other…).

Time window

The two pickers labelled `_startTime` and `_endTime` get literally substituted into your query text. Reference them anywhere you would use a time literal:

```
requests
| where timestamp >= _startTime and timestamp <= _endTime
| summarize count() by bin(timestamp, 5m)
```

Output

- **Grid** (default): sortable, filter-per-column, virtualized for large result sets.
- **Chart**: auto-detects time-series shape; falls back to bar/category.
- **JSON**: raw response, handy for debugging.
- **Export to Excel**: produces an `.xlsx` with the current grid and column types.

Troubleshooting

Symptom	Likely cause	Fix
`InsufficientAccessError (403)` on every query.	The service principal lacks `Monitoring Reader` on the AI resource.	Re-do the role assignment in Grant Azure permissions.

Symptom	Likely cause	Fix
Saved Queries shows only the built-ins.	`armId` missing from `vip_AppInsightsAppId`, or the SP can't list query packs in that resource group.	Add `armId`, and ensure `Monitoring Reader` covers the RG (or the AI resource alone is fine for analyticsItems).
Schema sidebar stays at "loading…".	App ID typo, or AI instance has zero ingested data, or token acquisition failed.	Check *Plug-in Trace Log* for `vip_azuremonitorquery`. The first call writes a token-acquisition trace.
Solution import fails with `0x8004f114` ("Webresource content size is too big").	The org's `maxuploadfilesize` is below the bundle size.	Bump it as shown in Prerequisites.
UI changes don't appear after upgrading.	Browser cached the old PCF bundle.	Hard refresh (Ctrl+F5) once. The bundle URL is fingerprinted on each managed import, so the cache will only stick for one session.

Source code, issues, and contributions: github.com/SweetsNSavories/Explorer-KQL · License: MIT.

Trademarks & attribution. Microsoft, Azure, Dynamics 365, Dataverse, Power Platform, Power Apps, Application Insights, and Azure Monitor are trademarks of the Microsoft group of companies. This project is an independent personal effort and is not affiliated with, sponsored by, or endorsed by Microsoft.